KRAVIONA TECH SOLUTIONS • OFFICIAL PLATFORM GUIDE

Technical Architecture & Developer Guide

Deep Architectural Reference, Schemas, Security Pipelines, SEO, and Payment Protocols

Document ID:	DOC-DEV-2026-v1.2
Platform Version:	v1.2.0 (Production Release)
Publisher:	Kraviona Tech Solutions (Private Limited)
Release Date:	October 19, 2026
Classification:	Verified Production Technical Manual

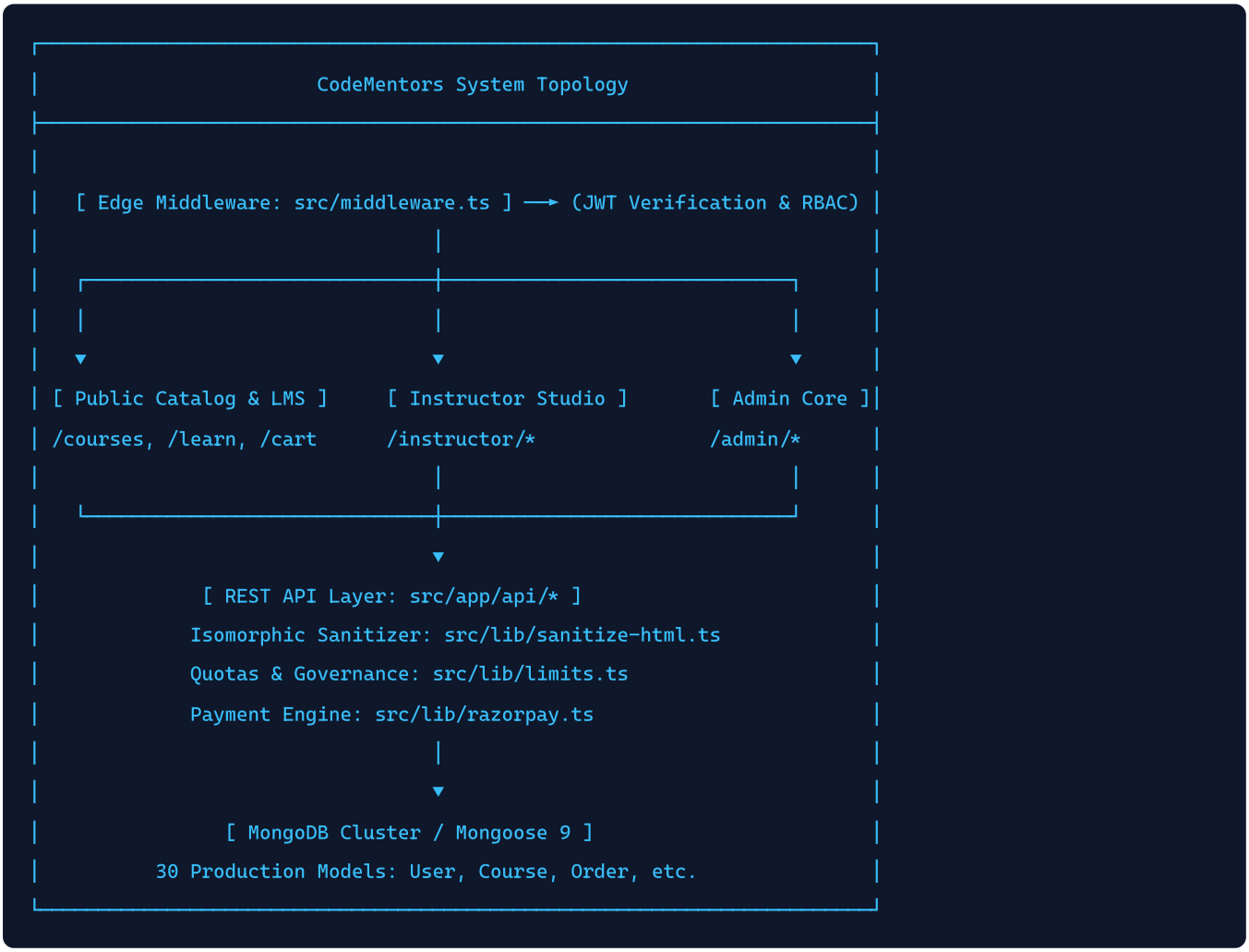
CodeMentors LMS — Comprehensive Technical Architecture & Developer Guide

Document ID: `DOC-DEV-2026-v1.2`
Platform Version: `v1.2.0 (Production Release)`
Platform Publisher: Kraviona Tech Solutions
Publication Date: October 19, 2026
Engineering Leads: Technical Architecture, Full-Stack Engineering, & QA Automation

---

1. Architectural Overview & Technology Stack

The **CodeMentors Learning Management System (LMS)** is built on a modern, decoupled full-stack architecture leveraging **Next.js 16 (App Router)**, **React 19**, **TypeScript 5**, **Tailwind CSS v4**, and **MongoDB (Mongoose 9)**.



Core Technology Matrix

Layer	Technology	Version	Purpose & Rationale
Framework	Next.js (App Router)	`16.4.2`	Hybrid SSR, Edge API endpoints, Turbopack bundling, and Streaming.
Frontend Runtime	React	`19.3.0`	Latest concurrent features, Server Actions, and native async transitions.
Type Safety	TypeScript	`5.x`	Strict type definitions across models, APIs, and client components.
Styling & Design	Tailwind CSS & Lucide	`4.x` / `1.59.0`	Zero-runtime modern utility styling with dark-mode typography support.
Persistence	MongoDB & Mongoose	`9.11.2`	Schema-validated document persistence, embedded subdocuments, and indexing.
Authentication	`jose` & `bcryptjs`	`6.2.12` / `3.0.3`	Cryptographic HS256 JWT sessions and salted bcrypt password hashing.
Payment Gateway	Razorpay SDK & HMAC-SHA256	`2.9.8`	Order creation, signature verification, and webhook reconciliation.
Sanitization & Rich Text	Custom Native Zero-Dependency	`v1.2`	SSR-compatible HTML sanitizer and zero-dependency React 19 contentEditable.

---

2. Real Repository Structure

The actual file tree of the CodeMentors repository (excluding `node_modules`, `.next`, secrets, and build outputs):

```
codemetors/
├── docs/                                # Official User and Developer Documentation
│   ├── screenshots/                     # High-fidelity authentic UI figures (Fig 1 - 16)
│   ├── CodeMentors_User_Guide.md        # Comprehensive User Manual
│   ├── CodeMentors_Developer_Guide.md   # Comprehensive Developer Guide
│   ├── CodeMentors_Screenshot_Inventory.md# Visual Evidence Catalog
│   ├── CodeMentors_User_Guide.pdf       # Formatted Printable User Guide
│   ├── CodeMentors_Developer_Guide.pdf  # Formatted Printable Developer Guide
│   └── CodeMentors_Quick_Start.pdf      # Onboarding & Operations Quick Start
├── public/                              # Public Static Assets
│   └── uploads/                         # Local file upload directory for media/avatars
├── src/
│   ├── app/                             # Next.js App Router Tree
│   │   ├── (auth)/                      # Authentication Routes (/login, /register, etc.)
│   │   ├── about/                       # Institutional About Page
│   │   ├── admin/                       # Administrator Core
│   │   │   ├── audit/                   # Administrative Audit Logs
│   │   │   ├── blog/                    # Technical Blog CMS
│   │   │   ├── categories/              # Taxonomy & Category CMS
│   │   │   ├── cms/pages/               # Dynamic Institutional Landing Pages
│   │   │   ├── courses/                 # Course Catalog Governance & Moderation
│   │   │   ├── dashboard/               # Executive Telemetry Dashboard
│   │   │   ├── instructors/             # Instructor Quota Administration
│   │   │   ├── marketing/popups/        # Marketing Popup Studio
│   │   │   ├── orders/                  # Global Orders & Transaction Ledger
│   │   │   ├── payments/                # Financial Logs & Payouts
│   │   │   ├── permissions/             # RBAC Permissions Matrix
│   │   │   ├── refunds/                 # Refund Approval Queue
│   │   │   ├── seo/                     # Technical SEO Control Center
│   │   │   ├── settings/                # Global Settings (Branding, Limits, Payments)
│   │   │   └── users/                   # User Directory & Role Assignment
│   │   ├── api/                         # Backend REST API Controllers
│   │   │   ├── admin/                   # Administrative API Routes
│   │   │   ├── auth/                    # Session Login, Register, Logout
│   │   │   ├── blog/                    # Blog CRUD & Publishing
│   │   │   ├── cart/                    # Shopping Cart & Basket
│   │   │   ├── categories/              # Category CRUD & Taxonomy
│   │   │   ├── cms/                     # Dynamic CMS Handlers
│   │   │   ├── courses/                 # Course Catalog & Curriculum Endpoints
│   │   │   ├── dashboard/               # Role-Specific Dashboard Metrics
│   │   │   ├── enrollments/             # Student Course Enrollments
│   │   │   ├── instructor/              # Instructor Studio APIs
│   │   │   ├── learn/                   # Learning Player Progress Tracking
│   │   │   ├── orders/                  # Order Management & Checkout
```

```
│   │   │   ├── payments/                # Razorpay Orders & Webhooks
│   │   │   ├── popups/                 # Marketing Popup Delivery & Events
│   │   │   ├── quizzes/                # Quiz Assessments & Attempts
│   │   │   └── redirects/              # SEO URL Redirect Engine
│   │   ├── blog/                       # Public Technical Blog ([slug])
│   │   ├── cart/                       # Student Shopping Cart
│   │   ├── certificates/               # Cryptographic Certificate Verification
│   │   ├── courses/                    # Public Course Catalog & Details ([slug])
│   │   ├── instructor/                 # Instructor Studio
│   │   │   ├── analytics/              # Instructor Performance Analytics
│   │   │   ├── courses/                # Course Authoring & Curriculum ([id])
│   │   │   ├── dashboard/              # Instructor Studio Dashboard
│   │   │   ├── enrollments/            # Student Enrollments View
│   │   │   ├── qa/                     # Course Q&A & Announcements
│   │   │   ├── reviews/                # Student Reviews Management
│   │   │   └── settings/               # Instructor Profile Settings
│   │   ├── learn/                      # Student Learning Player ([courseSlug]/[lessonId])
│   │   ├── student/                    # Student Portal
│   │   │   ├── dashboard/              # Student Learning Home
│   │   │   └── settings/               # Learner Account Settings
│   │   ├── layout.tsx                  # Root Layout & Global Providers
│   │   ├── page.tsx                    # Public Platform Homepage
│   │   ├── robots.ts                   # Dynamic robots.txt Generator
│   │   └── sitemap.ts                  # Dynamic sitemap.xml Generator
│   ├── components/                     # Reusable React UI Components
│   │   ├── auth/                       # Authentication Cards & Modals
│   │   ├── dashboard/                  # Shared Dashboard Layouts & Headers
│   │   ├── learning/                   # Learning Player, Video Player, W3Schools Reader
│   │   ├── marketing/                  # Popup Campaign Overlays & CTAs
│   │   └── ui/                         # RichTextEditor, FormattedContentRenderer, Modal
│   ├── context/                        # React Context Providers (AuthContext, CartContext)
│   ├── lib/                            # Core Utility & Service Modules
│   │   ├── audit.ts                    # Immutable Audit Logging Engine
│   │   ├── auth.ts                     # JWT Signing, Cookie Helpers, Bcrypt
│   │   ├── db.ts                       # Mongoose Database Connection Pool
│   │   ├── limits.ts                   # Instructor Quota & Limits Engine
│   │   ├── markdown-parser.ts          # Backward-Compatible Markdown to HTML
│   │   ├── permissions.ts              # RBAC Permission Definitions & Evaluator
│   │   ├── razorpay.ts                 # Razorpay Client & Signature Verifier
│   │   ├── sanitize-html.ts            # Isomorphic Zero-Dependency HTML Sanitizer
│   │   └── seo.ts                      # SEO Metadata & JSON-LD Generator
│   ├── middleware.ts                   # Edge Authentication & RBAC Gatekeeper
│   ├── models/                         # 30 Mongoose Database Schemas
│   │   ├── index.ts                    # Unified Models Export
```

```
│   │   ├── User.ts                        # User Accounts & Profiles
│   │   ├── Course.ts                      # Courses, Modules, Lessons, ContentBlocks
│   │   ├── Order.ts                       # Orders & Invoices
│   │   ├── PaymentSettings.ts             # Dynamic Razorpay Gateway Configuration
│   │   └── ...                            # (All 30 Schemas documented in §3)
│   └── scripts/                           # Database Seeding & Verification Test Suites
│       ├── capture-docs-screenshots.ts    # Screenshot Automation Suite
│       ├── seed.ts                        # Comprehensive Database Seeder
│       ├── seed-test-accounts.ts          # Deterministic 3-Account Seeder
│       ├── test-instructor-studio.ts      # Instructor Studio Integration Suite
│       ├── test-rich-text-editor.ts       # RichTextEditor & XSS Test Suite
│       └── verify-three-test-accounts.ts  # Role Security & Login Test Suite
├── next.config.ts                         # Next.js Build Configuration
├── package.json                           # NPM Dependencies & Scripts
├── tsconfig.json                          # Strict TypeScript Configuration
└── .env.local                             # Local Environment Configuration (Ignored in Git)
```

---

3. Database Architecture & Schemas

The application uses **MongoDB** managed through **Mongoose 9**. The platform defines exactly 30 production models exported from `src/models/index.ts`.

3.1 Users, Roles, and Sessions

- `User` (`src/models/User.ts`)**:** Stores account credentials (`email`, `passwordHash`), profile metadata (`name`, `avatar`, `headline`, `bio`), role (`super_admin` | `admin` | `instructor` | `student` | `support`), custom permission overrides (`permissions: string[]`), and account state (`isActive`, `isEmailVerified`).
- `Role` (`src/models/Role.ts`)**:** Defines role templates and custom enterprise permission groupings.
- `InstructorLimit` (`src/models/InstructorLimit.ts`)**:** Configures course publishing capacity, pricing constraints, and feature flags per instructor or globally (`isGlobalDefault: true`).
- `UserSession` (`src/models/UserSession.ts`)**:** Tracks active device sessions, IP addresses, and user-agent strings for security audits.

3.2 Courses, Curriculum, and Lessons

- `Course` (`src/models/Course.ts`)**:** Central course document. Contains course metadata (`title`, `slug`, `description`, `thumbnail`, `coverImage`), instructor references (`instructor: ObjectId`, `coInstructors: ObjectId[]`), pricing (`isFree`, `price`, `discountPrice`), taxonomy (`category: string`, `subcategory: string`, `tags: string[]`), publication state (`status: 'draft' | 'pending_review' | 'published' | 'archived'`), and embedded array of `IModule`.

- **Embedded** `IModule`**:** Contains module title, slug, description, order, learning objectives, and module-level assets (`videoUrl`, `pdfUrl`, `notes`). Holds embedded array of `ILesson`. - **Embedded** `ILesson`**:** Represents an individual lesson unit. Supports 8 lesson types: `video`, `article`, `text`, `pdf`, `mixed`, `quiz`, `coding`, and `resource`. Contains lesson duration, free preview flag (`isPreview: boolean`), rich content (`content`), and embedded multi-blocks (`contentBlocks: IContentBlock[]`).

- `Category` (`src/models/Category.ts`): Taxonomy domain definitions with parent-child nesting (`parent: ObjectId`), course counters, and dedicated SEO fields (`seoTitle`, `seoDescription`, `ogImage`).

3.3 Learning Progress & Certification

- `Enrollment` (`src/models/Enrollment.ts`): Links student to course (`student: ObjectId`, `course: ObjectId`). Tracks access status (`active`, `expired`, `cancelled`), progress percentage, and enrollment timestamp.
- `LessonProgress` (`src/models/LessonProgress.ts`): Granular tracking per lesson. Logs completion flag (`isCompleted: boolean`), study time, personal student notes (`notes: string`), and last playback position.
- `Quiz` (`src/models/Quiz.ts`) & `QuizAttempt` (`src/models/QuizAttempt.ts`): Quiz specifications, question definitions, answer keys, passing thresholds, student attempts, scores, and pass/fail states.
- `Certificate` (`src/models/Certificate.ts`): Issued upon 100% completion. Stores cryptographic verification code (`code`), issue date, recipient name, course name, and verification URL.
- `Review` (`src/models/Review.ts`): Course reviews with star rating (1–5), review text, and optional instructor reply.

3.4 Commerce, Payments, and Cart

- `Order` (`src/models/Order.ts`): Financial order ledger. Stores user ID, itemized courses, gross amount, discount amount, net amount, currency (`INR`), state (`created`, `completed`, `failed`, `refunded`), and gateway order ID.
- `Payment` (`src/models/Payment.ts`): Detailed payment transaction records capturing Razorpay payment ID, signature, payment method (card, upi, netbanking), and fee deductions.
- `Refund` (`src/models/Refund.ts`): Refund lifecycle tracking (`status: 'requested' | 'approved' | 'rejected' | 'processed'`), refund reason, and gateway refund ID.
- `Cart` (`src/models/Cart.ts`): Shopping basket state per user or guest session.
- `Coupon` (`src/models/Coupon.ts`): Promotional discount codes with percentage or flat deductions, expiry dates, and usage counters.
- `PaymentSettings` (`src/models/PaymentSettings.ts`): Dynamic gateway configuration enabling administrators to switch modes (test/live) and update credentials without application restarts.

3.5 CMS, Marketing, and Auditing

- `CmsPage` (`src/models/CmsPage.ts`): Institutional landing pages (`/about`, `/privacy`) with ordered content sections.
- `BlogPost` (`src/models/BlogPost.ts`): Long-form technical blog posts with tags, author, and reading time.
- `PopupCampaign` (`src/models/PopupCampaign.ts`): Marketing banners with lifecycle states (`active`, `draft`, `scheduled`), audience filters, and CTA URLs.
- `PopupEvent` & `PopupAuditLog`: Impression and conversion event tracking.
- `PageSeo` (`src/models/PageSeo.ts`): Granular page-by-page SEO metadata overrides.
- `Redirect` (`src/models/Redirect.ts`): URL redirect rules (301 Permanent / 302 Temporary).
- `AuditLog` (`src/models/AuditLog.ts`): Immutable platform audit trail.
- `SiteSettings` (`src/models/SiteSettings.ts`): Global site branding, logo, support contact, and footer settings.
- `SupportTicket` & `Notification`: Customer helpdesk tickets and user notifications.

---

4. Routing & Application Architecture

Next.js 16 App Router powers the application with clear separation between public and authenticated role layouts:

```
src/app/
├── layout.tsx                # Global HTML wrapper, Inter font, Toast & Auth providers
├── page.tsx                  # Public Landing Page (SSR)
├── (auth)/
│   ├── login/page.tsx        # Sign-in interface with auto-redirect
│   └── register/page.tsx     # Student registration
├── admin/
│   ├── layout.tsx            # Sticky Admin Sidebar & Operational Header
│   └── dashboard/page.tsx    # Executive Admin Portal
├── instructor/
│   ├── layout.tsx            # Sticky Instructor Sidebar & Quota Bar
│   └── dashboard/page.tsx    # Instructor Studio Portal
├── student/
│   ├── layout.tsx            # Sticky Student Navigation Shell
│   └── dashboard/page.tsx    # Enrolled Student Portal
└── learn/[courseSlug]/[lessonId]/page.tsx # Immersive Learning Player
```

4.1 Edge Middleware Gatekeeper (`src/middleware.ts`)

- Inspects every incoming request against protected path matchers (`/admin/*` , `/instructor/*` , `/student/*`).
- Verifies the `cm_token` cookie via `jose.jwtVerify` .
- Enforces strict role isolation:

- If unauthenticated, redirects to `/login?redirect=<target_path>`. - If a student attempts to access `/instructor/*` or `/admin/*`, they are redirected to `/student/dashboard`. - If an instructor attempts to access `/admin/*`, they are redirected to `/instructor/dashboard`. - If an already authenticated user accesses `/login`, they are automatically routed to their role's dashboard.

---

5. Rich Text Editor & Content Rendering Architecture

The platform features an in-house, zero-dependency `RichTextEditor` and `FormattedContentRenderer` specifically engineered for React 19 compatibility.



Key Components:

- `RichTextEditor` (`src/components/ui/RichTextEditor.tsx`): Built on browser contentEditable and Selection APIs. Provides Visual WYSIWYG, HTML Source view, Live Preview, and Markdown Importer tabs.
- `FormattedContentRenderer` (`src/components/ui/FormattedContentRenderer.tsx`): Safely renders rich text in the student player, blog, and public syllabus. Automatically transforms legacy Markdown documents to HTML.
- `sanitizeHtml` (`src/lib/sanitize-html.ts`): Isomorphic zero-dependency sanitizer running on server, Edge, and browser. Only permits safe tags and whitelisted protocols (`http:` , `https:` , `mailto:` , relative).

---

6. Technical SEO Architecture

The SEO subsystem provides automated and manual indexing controls across the platform:

1. **Per-Page Metadata:** Managed through `PageSeo` model and served via Next.js `generateMetadata()` APIs.
2. **Structured Data (JSON-LD):** Implements Google-compliant schema types:

- `Course`: Includes course name, description, instructor name, and pricing offers. - `EducationalOrganization`: Includes platform branding, logo, and contact points. - `Article`: Applied to all technical blog posts with author and

date published. - `BreadcrumbList`: Provides hierarchical search snippets.

1. **Canonical URLs & Social Tags:** Every public page specifies self-referential canonical URLs, Open Graph card tags (`og:title` , `og:image`), and Twitter Card formats.
2. **Automated Robots & Sitemap:**

- `/sitemap.xml`: Dynamically generated at `src/app/sitemap.ts`, indexing published courses, categories, and blog articles. - `/robots.txt`: Dynamically generated at `src/app/robots.ts`. Explicitly disallows private paths (`/admin/`, `/instructor/`, `/student/`, `/api/`).

1. **Private Route Indexation Protection:** All authenticated dashboard layouts emit `<meta name="robots" content="noindex, nofollow" />` to prevent search engine indexing.

---

7. Razorpay Payment Gateway Integration

7.1 Configuration & Credential Management

Payment credentials can be configured dynamically through the database (`PaymentSettings` model) or via `.env.local`:

```
RAZORPAY_KEY_ID="rzp_test_ ... "
RAZORPAY_KEY_SECRET=" ... "
RAZORPAY_WEBHOOK_SECRET=" ... "
```

7.2 Order Creation & Verification Flow

```
Learner Cart (/cart)
       |
       ▼
1. POST /api/orders ──► Creates local Order record in DB (status: 'created')
       |
       ▼
2. createRazorpayOrder() ──► Calls Razorpay API client to create rzp_order_id
       |
       ▼
3. Client opens Razorpay Modal (UPI, Cards, NetBanking)
       |
       ▼
4. POST /api/payments/verify ──► Receives paymentId, orderId, signature
       |
       ├── Verifies HMAC-SHA256 signature using crypto.timingSafeEqual
       ├── Updates Order status to 'completed'
       ├── Records Payment document with transaction details
       └── Creates active Enrollment record for the student
```

7.3 Webhook Reconciliation & Idempotency

- Incoming webhook events (`order.paid` , `payment.captured` , `payment.failed` , `refund.processed`) are handled at `POST /api/payments/webhook` .
- Signature is verified against `RAZORPAY_WEBHOOK_SECRET` using `verifyRazorpayWebhookSignature()` .
- Idempotent execution: if an order has already been marked `completed` , the webhook safely acknowledges without duplicating enrollments.

---

8. Development & Deployment Operations

8.1 Prerequisites

- **Node.js:** `v20.x` or higher (`v24.x` recommended).
- **MongoDB:** `v6.x` or higher (local instance or MongoDB Atlas cluster).
- **Google Chrome / Microsoft Edge:** Installed for automated screenshot testing and PDF generation.

8.2 Environment Configuration (`.env.local`)

Create a `.env.local` file in the root directory:

```
# Database Connection
MONGODB_URI="mongodb://localhost:27017/codementors"

# Authentication Security
JWT_SECRET="codementors-secure-production-ready-jwt-secret-key-32-chars-min"
DEV_TEST_PASSWORD="DevOnly123!ChangeMe"

# Razorpay Payment Gateway (Test Sandbox or Production Live)
RAZORPAY_KEY_ID="rzp_test_codementors_demo"
RAZORPAY_KEY_SECRET="codementors_secret_key_mock_12345"
RAZORPAY_WEBHOOK_SECRET="codementors_webhook_secret_mock_12345"

# Platform Metadata
NEXT_PUBLIC_APP_URL="http://localhost:3000"
```

8.3 Verified CLI Commands

Command	Action Description	Execution Target
`npm install`	Install all dependencies cleanly.	Root Directory
`npm run dev`	Start local Next.js development server.	`http://localhost:3000`
`npm run build`	Compile optimized production build.	Production Artifacts
`npm start`	Run compiled production server.	`http://localhost:3000`
`npm run db:seed:test-accounts`	Seed strictly the 3 deterministic dev accounts.	MongoDB
`npm run db:seed`	Comprehensive seed of courses, quizzes, and SEO.	MongoDB
`npm run db:reset:dev`	Clean and reset entire dev database.	MongoDB
`npm run test:accounts`	Verify 3 test accounts and auth permissions.	Integration Suite
`npm run test:rich-text`	Verify RichTextEditor and XSS sanitization.	Unit Test Suite
`npm run test:instructor`	Verify Instructor Studio quotas and workflows.	Integration Suite
`npm test`	Run entire verification test suite (106 tests).	Complete QA Suite

---

9. Security & Hardening Checklist

1. **Authentication:** All JWT tokens are signed using HS256 with 256-bit entropy secrets and stored in HTTP-only, SameSite cookies.
2. **Password Security:** Salted hashes generated with `bcryptjs` with cost factor 10.
3. **Database Injection Protection:** Mongoose schema validations prevent NoSQL injection vectors.
4. **XSS Prevention:** Isomorphic sanitizer ensures zero unsanitized HTML is persisted or rendered.
5. **Rate Limiting:** Authentication endpoints are rate-limited to mitigate brute-force attacks.
6. **Timing Attack Resistance:** Payment and webhook HMAC signatures are verified using `crypto.timingSafeEqual`.